

YAZILIM TASARIM DÖKÜMANI (SDD)

Web Tabanlı Tank Savaşı Oyunu

Grup Numarası: 1

Yusuf Tarlan (150619027)
Yusuf Eren ÇELEBİ (170424826)
Muhammed Kızıldağ (170423963)
Mhd Diaa Alsebai (170423954)
Nuri Sarı (170424517)

4 Mayıs 2026

İçindekiler

1	Giriş	1
1.1	Projenin Amacı ve Kapsamı	1
1.2	Terimler ve Kısaltmalar	1
2	Sistem Mimarisi	2
2.1	Meta Server (Lobi Katmanı) Mimarisi	2
2.2	Game Server (Mekanik Katmanı) Mimarisi	3
2.3	İstemci-Sunucu İletişim Akışı (Handover Process)	3
3	Frontend Tasarımı	3
3.1	Teknoloji Yığını ve Modüler Yapı	3
3.2	Render Hattı (Rendering Pipeline)	4
3.3	Gelişmiş Ağ Senkronizasyon Teknikleri	4
3.4	Varlık (Asset) Yönetimi ve Yükleme	4
3.5	Kullanıcı Deneyimi (HUD ve Menüler)	4
4	Backend Tasarımı	5
4.1	RAM Tabanlı Veri Yapıları ve Şemalar (Data Schemas)	5
4.1.1	Oturum ve Kimlik Yönetimi Verileri	5
4.1.2	Çöp Toplama ve Bellek Temizleme Politikası	5
4.2	REST API Uç Noktaları (Endpoints)	6
4.2.1	Oda Oluşturma (Create Room)	6
4.2.2	Aktif Odaları Listeleme (Get Rooms)	6
5	Oyun Mekaniği ve Fizik	7
5.1	Tank Hareket Mekaniği	7
5.2	Mermi Fiziği ve Balistik	7
5.3	Çarpışma Algılama (Collision Detection)	8
5.4	Hasar ve Sağlık Sistemi (Health System)	8
5.5	Dinamik Harita ve Döngüsel Fizik	8
5.6	Özel Güçler (Power-Ups) ve Nesne Fiziği	8
5.7	Gelişmiş Mermi ve Balistik Modelleri	9
5.8	Hasar ve Respawn Mekanizması	9

1 Giriş

1.1 Projenin Amacı ve Kapsamı

Bu projenin temel amacı; modern web teknolojileri (HTML5, WebSocket, Node.js) kullanılarak, eklenti veya indirme gerektirmeyen, tarayıcı üzerinden doğrudan oynanabilen gerçek zamanlı (real-time) çok oyunculu bir 2D tank savaşı oyunu geliştirmektir. Proje, sadece eğlence odaklı bir oyun olmanın ötesinde, ağ senkronizasyonu ve durum (state) yönetimi gibi ileri seviye bilgisayar mühendisliği problemlerine ölçeklenebilir çözümler getirmeyi hedefler.

Kapsam Dahilinde olanlar:

- Geçici oturum (ephemeral session) mantığıyla çalışan, veritabanı bağımsız (RAM tabanlı) kullanıcı ve lobi yönetimi.
- HTTP tabanlı bir "Meta Server" ile oda listeleme ve eşleştirme (matchmaking) sistemi.
- İstemci-sunucu mimarisini ayıran, yetkili sunucu (Authoritative Server) altyapısı.
- WebSocket üzerinden düşük gecikmeli veri akışı sağlanması.

Kapsam Dışında Bırakılanlar:

- Kalıcı kullanıcı hesapları, şifreleme ve ilişkisel bir veritabanı (RDBMS) yönetimi (sistem tamamen durumsuz/stateless çalışacak şekilde tasarlanmıştır).
- 3D grafik işleme ve karmaşık yeryüzü fiziği (z eksenini).

1.2 Terimler ve Kısaltmalar

Döküman boyunca sıklıkla kullanılacak olan teknik terimlerin ve sistem bileşenlerinin tanımları aşağıda listelenmiştir:

RTT (Round Trip Time): İstemciden sunucuya gönderilen bir veri paketinin... toplam süredir. Genellikle *ms* cinsinden ölçülür.

Tick Rate (Tik Oranı): Oyun sunucusunun (Game Server) oyun dünyasını ve fiziksel olayları saniyede kaç kez güncellediğini ifade eden frekans değeridir. Sistemimizde Tick Rate 60 Hz olarak belirlenmiş olup, her bir karenin (frame) işleme süresi (Delta Time, Δt) yaklaşık olarak şu kadardır:

$$\Delta t = \frac{1000 \text{ ms}}{60} \approx 16.66 \text{ ms}$$

Authoritative Server (Yetkili Sunucu): Dağıtık oyun mimarilerinde "tek gerçek" (single source of truth) olarak kabul edilen sunucu modelidir. İstemciler sadece "ateş et", "sağa dön" gibi girdileri (input) gönderir. Çarpışma, hız, hasar alma gibi tüm kritik hesaplamalar sunucuda yapılır ve sonuçlar istemcilere yayınlanır (broadcast).

Game Loop (Oyun Döngüsü): Oyun motorunun ve mekanik sunucusunun kalbini oluşturan, durmaksızın çalışan döngüdür. Her iterasyonda sırasıyla; girdiler okunur, fizik (hız, yön, çarpışma) güncellenir ve yeni durum render edilmek (çizilmek) üzere hazırlanır.

Meta Server (Lobi Sunucusu): Oyunun doğrudan oynanmadığı; oyuncu girişi, oda oluşturma ve sunucu atama (matchmaking) gibi işlemlerin standart HTTP (Express.js) protokolüyle yönetildiği durumsuz (stateless) katmandır.

Ephemeral Data (Geçici Veri): Sabit diske veya bir veritabanına yazılmayan, sadece Node.js sürecinin (process) RAM’inde yaşayan ve oyuncunun bağlantısı koptuğunda yok olan veri yapılarıdır.

2 Sistem Mimarisi

Bu proje, performans darboğazlarını engellemek ve sistem kaynaklarını optimize etmek amacıyla mikroservis mimarisinden ilham alan ”İkili Sunucu” (Dual-Server) yapısı üzerine inşa edilmiştir. Sistem; oyuncu eşleştirme işlemlerini yöneten durumsuz (stateless) Meta Server ve gerçek zamanlı oyun fiziğini hesaplayan durumlu (stateful) Game Server olmak üzere iki ana bileşenden oluşur.

2.1 Meta Server (Lobi Katmanı) Mimarisi

Meta Server, sistemin dış dünyaya açılan ilk kapısıdır ve oyuncuların oyun (savaş) alanına aktarılmadan önceki tüm eşleştirme, lobi ve kimlik doğrulama süreçlerini yöneten HTTP tabanlı bir mikroservis olarak tasarlanmıştır. Bu katman, sürekli açık bir socket bağlantısının getirdiği donanım yükünü (overhead) minimize etmek amacıyla İstek-Cevap (Request-Response) döngüsüyle çalışacak şekilde kurgulanmıştır.

Teknoloji Yığını ve İletişim Protokolü

Sunucu, Node.js çalışma zamanı (runtime) üzerinde **Express.js** web iskeleti (framework) kullanılarak inşa edilmiştir. İstemci (oyuncunun tarayıcısı) ile sunucu arasındaki iletişim, standart RESTful HTTP/HTTPS protokolleri üzerinden sağlanmaktadır. Bu tercih, uygulamanın Load Balancer (Yük Dengeleyici) arkasında kolayca yatay olarak ölçeklenebilmesine (horizontal scaling) olanak tanır.

Hibrit Durum (Hybrid State) Yönetimi ve Geçici Oturumlar

Meta Server, kimlik doğrulama süreçleri açısından *Durumsuz (Stateless)*, lobi ve oda yönetimi açısından ise *Durumlu (Stateful)* bir hibrit mimariye sahiptir:

- **Kimlik Doğrulama (Stateless):** Geleneksel şifreli ve veritabanı bağımlı üyelik sistemleri yerine, hıza odaklı ”Geçici Oturum” (Ephemeral Session) mimarisi kullanılmıştır. Kullanıcılar sadece bir kullanıcı adı ile giriş yapar ve sunucu bu kullanıcıya sistem içi yetkilendirmede kullanılacak tek kullanımlık bir JSON Web Token (JWT) tanımlar.
- **Oda Yönetimi (Stateful):** Sunucu, aktif oda listesini harici bir veritabanı (RDBMS veya NoSQL) yerine doğrudan kendi belleğinde (RAM) JavaScript Map veri yapısı içerisinde tutar. Bu sayede disk I/O işlemlerinden kaynaklanan gecikmeler ortadan kaldırılarak verilere $O(1)$ zaman karmaşıklığında erişim sağlanır.

Temel API Uç Noktaları (Endpoints) ve İşlevleri

Lobi mimarisi, aşağıdaki temel rotalar üzerinden istemciye hizmet verir:

- **POST /api/login:** Kullanıcı adını kabul eder, doğrular ve oturum biletini (JWT) döner.
- **GET /api/rooms:** RAM üzerinde tutulan aktif ve "bekliyor" statüsündeki oda listesini JSON formatında istemciye iletir.
- **POST /room/create & POST /room/join:** Yeni oda oluşturma ve mevcut bir odaya katılma işlemlerini yönetir. Odanın maksimum oyuncu kapasitesi (N_{max}) gibi ön koşul doğrulamaları bu uç noktalarda yapılır.
- **GET /api/room/:roomId:** Oyuncu odaya (bekleme ekranına) girdiğinde, diğer oyuncuların gelip gelmediğini görmek için kullanılır. Frontend, oyun başlayana kadar her 2 saniyede bir bu API'ye istek atarak (Polling) odadaki güncel oyuncu listesini çeker.
- **POST /api/room/start:** Odadaki gameMaster (Kurucu) "Oyunu Başlat" butonuna bastığında tetiklenir. Odayı in-game statüsüne alır ve oyuncuların WebSocket (Game Server) tarafına geçmesi için gerekli olan biletleri (Match Token) üretilip geri döner.

Performans Hedefleri

Veritabanı okuma/yazma işlemlerinin mimariden çıkarılması sayesinde, bir HTTP isteğinin toplam işleme süresi (T_{total}), yalnızca ağ gecikmesi ($T_{network}$) ve Express.js middleware işleme süresi ($T_{processing}$) ile sınırlıdır:

$$T_{total} = T_{network} + T_{processing}$$

Burada $T_{processing}$ süresinin < 5 ms altında tutulması hedeflenmiş olup, bu durum istemci tarafında lobinin anlık (gerçek zamanlıya yakın) güncelleniyormuş hissi vermesini sağlamaktadır.

2.2 Game Server (Mekanik Katmanı) Mimarisi

2.3 İstemci-Sunucu İletişim Akışı (Handover Process)

3 Frontend Tasarımı

Frontend mimarisi, tarayıcı kaynaklarını verimli kullanarak akıcı bir oyun deneyimi sunmak üzere tasarlanmıştır. Bu bölüm; teknoloji seçimi, render döngüsü ve ağ senkronizasyonu detaylarını kapsar.

3.1 Teknoloji Yığını ve Modüler Yapı

İstemci tarafı, modern web standartları ve modüler JavaScript yapısı üzerine kuruludur:

- **HTML5 Canvas API:** Donanım hızlandırmalı (Hardware Accelerated) 2D çizim için kullanılır. Her karede (60 fps) tüm oyun nesneleri bu katman üzerinde render edilir.
- **Socket.io-client:** Sunucu ile düşük gecikmeli, olay tabanlı iletişim sağlar.

- **Tailwind CSS:** Oyun dışı arayüzlerin (lobiler, menüler) hızlı ve duyarlı tasarımı için kullanılır.
- **State Store (Local):** Oyunun anlık durumunu (`gameState`) bellekte tutan, sunucudan gelen paketlerle güncellenen merkezi bir nesne yapısıdır.

3.2 Render Hattı (Rendering Pipeline)

Görüntü kalitesini artırmak ve işlemci yükünü azaltmak için katmanlı bir çizim stratejisi izlenir:

- **Arka Plan Katmanı:** Statik harita verileri ve engeller bir kez çizilip "off-screen canvas" üzerinde önbellege alınır.
- **Dinamik Katman:** Hareketli tanklar, mermiler ve özel güçler her karede yeniden çizilir.
- **Parçacık Sistemi (Particle System):** Patlamalar ve motor dumanı gibi efektler için kısa ömürlü nesnelerden oluşan hafif bir alt sistem çalışır.
- **Arayüz (HUD) Katmanı:** HTML/CSS kullanılarak Canvas üzerine bindirilen (overlay), anlık veri gösteren katmandır.

3.3 Gelişmiş Ağ Senkronizasyon Teknikleri

İstemci, sunucudaki fizik döngüsüyle senkronize kalmak için şu algoritmaları çalıştırır:

- **Client-Side Prediction (Tahmin):** Kullanıcı hareket girdiğini gönderdiği anda, yerel tank sunucudan onay beklemeden hareket ettirilir. Bu, *RTT* kaynaklı gecikme hissini yok eder.
- **Server Reconciliation (Mutabakat):** Sunucudan gelen kesin konum verisi, yerel tahminlerle uyuşmuyorsa, tankın konumu "yumuşak bir geçişle" sunucunun bildirdiği noktaya düzeltilir.
- **Entity Interpolation (Enterpolasyon):** Diğer oyuncuların verileri ağ paketleri arasındaki boşluklarda (16.66 ms) doğrusal (linear) olarak tahmin edilir. Böylece rakiplerin hareketi kesikli değil, akıcı görünür.

3.4 Varlık (Asset) Yönetimi ve Yükleme

Oyun başlamadan önce gerekli tüm grafik varlıkları belleğe alınır:

- **Preloader:** Resim ve ses dosyaları yüklenmeden "Başla" butonu aktif edilmez.
- **Sprite Sheets:** Tüm tank varyasyonları ve mermi tipleri tek bir büyük resim dosyasında (atlas) tutulur. Bu, GPU üzerindeki "draw call" sayısını azaltarak performans artırır.

3.5 Kullanıcı Deneyimi (HUD ve Menüler)

- **Leaderboard:** Ekranın köşesinde, WebSocket üzerinden gelen "kill-count" verileriyle anlık güncellenen liste.

- **Dinamik Can Barları:** Her tankın üzerinde, tankın koordinatlarını takip eden mini ilerleme çubukları.
- **Lag Göstergesi:** İstemci ve sunucu arasındaki RTT süresini görsel olarak kullanıcıya sunan sinyal göstergesi.

4 Backend Tasarımı

4.1 RAM Tabanlı Veri Yapıları ve Şemalar (Data Schemas)

Sistemde harici bir veritabanı (RDBMS veya NoSQL) bulunmadığından, uygulamanın tüm durumu (state) Node.js sürecinin belleğinde (RAM) JavaScript'in yerleşik veri yapıları olan **Map** ve **Set** objeleri ile yönetilmektedir. Bu tercih, verilere ortalama $\mathcal{O}(1)$ zaman karmaşıklığında erişilmesini sağlar.

Sistemdeki temel veri yapıları üç ana gruba ayrılır: Oturum Verileri, Lobi/Oda Verileri ve Oyun İçi Dinamik Veriler.

4.1.1 Oturum ve Kimlik Yönetimi Verileri

Kullanıcı giriş yaptığında oluşan ve oyuncunun sistemdeki varlığını temsil eden veri yapılarıdır.

A. activeUsernames (Veri Tipi: Set)

- **Amacı:** Sisteme giriş yapmış kullanıcı adlarının benzersizlik (unique) kontrolünü milisaniyeler içinde yapmak.
- **İçerik:** ["yusuf", "eren_tank", "player1"]

B. activeSessions (Veri Tipi: Map)

- **Amacı:** Kullanıcının sahip olduğu oturum bileti (Token) ile kimlik bilgilerini eşleştirmek. Yetkilendirme (Authorization) işlemlerinde kullanılır.
- **Anahtar (Key):** Token (UUID formatında, örn: "1b9d6bcd...")
- **Değer (Value) Şeması:**

```
{
  "username": "yusuf",
  "joinedAt": 1714300000000, // Giriş yapılan Unix zaman damgası
  "currentRoom": "room_a1b2" // Oyuncu bir odadaysa ID'si, değilse null
}
```

4.1.2 Çöp Toplama ve Bellek Temizleme Politikası

RAM şişmesini engellemek için sistemde iki temel kural uygulanır:

1. **Bağlantı Kopması (Disconnect):** Socket veya HTTP üzerinden bir oyuncu ayrıldığında, oyuncunun bilgileri **activeUsernames** ve **activeSessions** üzerinden $\mathcal{O}(1)$ maliyetle anında silinir (Örn: `activeSessions.delete(token)`).

2. **Zombi Odalar:** Durumu "waiting" olan ancak $T_{su.an} - T_{createdAt} > 3600$ sn (1 saat) şartını sağlayan odalar, `setInterval` ile çalışan bir arka plan temizleyicisi tarafından periyodik olarak bellekten atılır.

4.2 REST API Uç Noktaları (Endpoints)

Meta Server üzerinde çalışan HTTP rotalarının mantıksal işleyişi, beklediği parametreler ve döndürdüğü JSON şemaları aşağıda detaylandırılmıştır.

4.2.1 Oda Oluşturma (Create Room)

Bu uç nokta, oyuncunun yeni bir savaş alanı (oda) başlatmasını sağlar. İstek başarılı olduğunda, RAM üzerinde yeni bir Room nesnesi oluşturulur.

- **URL:** `/api/room/create`
- **Metod:** POST
- **Yetkilendirme (Auth):** Bearer Token (JWT) gerektirir.

Gelen İstek (Request Payload): İstemci, oluşturulacak odanın temel ayarlarını gönderir.

```
{
  "roomName": "Acemiler Giremez",
  "maxPlayers": 4,
  "mapId": "desert_01"
}
```

Başarılı Yanıt (Success Response - 200 OK): Sunucu, RAM'de odayı oluşturur ve benzersiz bir ID atayarak yanıt döner.

```
{
  "success": true,
  "roomId": "room_a7b9c",
  "message": "Oda başarıyla oluşturuldu."
}
```

Hata Durumları (Error Responses):

- **400 Bad Request:** Eksik veya hatalı parametre (Örn: `maxPlayers` değeri 2'den küçükse).
- **401 Unauthorized:** Geçersiz veya süresi dolmuş JWT.

4.2.2 Aktif Odaları Listeleme (Get Rooms)

Lobi ekranındaki oyunculara, mevcut odaların durumunu iletmek için kullanılır.

- **URL:** `/api/rooms`
- **Metod:** GET

Başarılı Yanıt (Success Response - 200 OK): RAM’de tutulan `rooms` Map nesnesi, JSON formatına serileştirilerek döndürülür. Sadece durumu `"waiting"` (oyun başlamamış) olan odalar filtrelenir.

```
{
  "activeRooms": [
    {
      "roomId": "room_a7b9c",
      "roomName": "Acemiler Giremez",
      "currentPlayers": 1,
      "maxPlayers": 4,
      "gameMaster": "yusuf"
    }
  ]
}
```

5 Oyun Mekaniği ve Fizik

Bu bölüm, tankların hareket kabiliyetlerini, mermi fiziğini ve oyun dünyasındaki nesnelerin birbirleriyle olan etkileşimlerini yöneten matematiksel modelleri ve algoritmaları detaylandırır.

5.1 Tank Hareket Mekaniği

Tanklar, 2B düzlemde vektörel bir hareket modeline sahiptir. Her tankın anlık durumu; pozisyon (x, y) ve gövde açısı (θ) bileşenleri ile tanımlanır.

- **Doğrusal Hareket:** Sunucu, istemciden gelen "ileri" veya "geri" girdi paketini aldığı anda tankın yeni pozisyonunu şu trigonometrik formüllerle hesaplar:

$$x_{yeni} = x_{eski} + (v \cdot \cos(\theta) \cdot \Delta t)$$

$$y_{yeni} = y_{eski} + (v \cdot \sin(\theta) \cdot \Delta t)$$

Burada v tankın doğrusal hızını, Δt ise her bir sunucu tikinin (yaklaşık 16.66 ms) süresini temsil eder.

- **Açısal Hareket (Rotasyon):** Tankın kendi eksenini etrafında dönme hızı ω (radyan/sn) sabittir. Dönüş girdisi alındığında açısal değişim şu şekilde işlenir:

$$\theta_{yeni} = \theta_{eski} + (\omega \cdot \text{input}_{yon} \cdot \Delta t)$$

5.2 Mermi Fiziği ve Balistik

Mermiler, ateşlendiği andaki tankın doğrultusunda fırlatılan ve yerçekiminden etkilenmeyen (üstten görünüm nedeniyle) doğrusal nesnelerdir.

- **İnitialization (Başlatma):** Mermi nesnesi, tankın namlu ucu koordinatlarında (x_{namlu}, y_{namlu}) oluşturulur ve tankın o anki θ açısına sahip olur.
- **Kinematik Güncelleme:** Mermiler, tank hızından bağımsız sabit bir $v_{projectile}$ hızına sahiptir. Sunucu her tick içerisinde mermi konumunu günceller ve harita sınırları dışına çıkan mermileri bellekten ($O(1)$ maliyetle) siler.

5.3 Çarpışma Algılama (Collision Detection)

Sistemde performans ve hassasiyet dengesini korumak adına iki katmanlı bir çarpışma kontrolü uygulanmaktadır:

1. **AABB (Axis-Aligned Bounding Box):** Tanklar ve dikdörtgen engeller için kullanılır. İki nesnenin kesişimi, x ve y eksenleri üzerindeki izdüşümlerinin çakışmasıyla kontrol edilir.
2. **Daire-Kutu Çarpışması (Mermiler için):** Mermiler küçük olduğu için daire olarak kabul edilir. Mermi merkezi ile en yakın engel noktası arasındaki Öklid uzaklığı (d), mermi yarıçapından (r) küçükse çarpışma tetiklenir:

$$\sqrt{(x_{mermi} - x_{engel})^2 + (y_{mermi} - y_{engel})^2} < r$$

5.4 Hasar ve Sağlık Sistemi (Health System)

Bir mermi bir tanka çarptığında sunucu şu mantıksal sırayı izler:

- Mermi nesnesi sunucu belleğinden silinir.
- Hedef tankın **health** değişkeninden merminin **damage** puanı düşülür.
- Eğer $health \leq 0$ ise; tank yok edilmiş olarak işaretlenir, saldırgan oyuncunun skoru artırılır ve yeni haritaya geçilir.

5.5 Dinamik Harita ve Döngüsel Fizik

Oyun, statik bir harita yerine her tur başında veya oyuncu ölümlerinde tetiklenen rastgele harita sistemine sahiptir.

- **Harita Değişimi:** Bir oyuncu imha edildiğinde ($health \leq 0$), sunucu mevcut oturum için yeni bir rastgele engel dizilimi ($\mathcal{M}_{new}$) üretir ve tüm istemcilere yeni koordinat verilerini yayınlar.
- **Sınır Kontrolü:** Harita sınırları katı bariyerler olarak tanımlanmıştır. Tankın yeni hesaplanan koordinatı harita genişliği W veya yüksekliği H dışındaysa, hareket vektörü sınıra sabitlenir.

5.6 Özel Güçler (Power-Ups) ve Nesne Fiziği

Haritanın rastgele koordinatlarında, tankın ateşleme ve savunma özelliklerini geçici olarak değiştiren özel güçlendirmeler bulunur.

- **Güdümlü Füze (Homing Missile):** Mermi vektörü $\vec{v}$, her tick'te en yakın düşman tankının $\vec{p}_{target}$ konumuna doğru belirli bir α yönelme katsayısı ile güncellenir:

$$\vec{v}_{next} = \text{Normalize}(\vec{v}_{curr} + (\vec{p}_{target} - \vec{p}_{curr}) \cdot \alpha) \cdot v_{max}$$

- **Taramalı Tüfek (Rapid Fire):** Ateşleme döngüsündeki $\Delta t_{cooldown}$ süresi %75 oranında azaltılarak saniyedeki mermi çıkış hızı artırılır.

- **Duvar Delen Mermi (Ghost Bullet):** Çarpışma algılama algoritmasında mermi'nin `isGhost` bayrağı kontrol edilir. Eğer aktifse, mermi statik engel (AABB) kontrollerini pas geçer ve sadece tank nesneleriyle etkileşime girer.
- **Hiper Motor (Turbo Drive):** Tankın doğrusal hareket hızını (v) ve açısal dönüş hızını (ω) geçici bir süreliğine %50 oranında artıran bir güçlendiricidir. Bu mod aktifken, sunucu tarafındaki fizik döngüsünde tankın birim zamandaki yer değiştirme katsayısı güncellenir:

$$v_{turbo} = v_{base} \cdot 1.5$$

$$\omega_{turbo} = \omega_{base} \cdot 1.5$$

Bu özellik, oyuncuya hem saldırıdan kaçma hem de rakipleri çevreleme (flanking) konusunda stratejik avantaj sağlar.

5.7 Gelişmiş Mermi ve Balistik Modelleri

Mermi türlerine göre fizik motoru farklı çarpışma ve yayılma algoritmaları çalıştırır:

- **Alan Hasarlı Bomba (AOE Explosion):** Mermi bir yüzeye çarptığında $R_{explosion}$ yarıçapında bir etki alanı oluşturur. Hasar, merkeze olan uzaklık (d) ile ters orantılıdır:

$$\text{Damage} = \text{BaseDamage} \cdot \left(1 - \frac{d}{R_{explosion}}\right)$$

- **Şarapnel Bombası (Cluster Bomb):** Çarpışma anında mermi yok edilir ve çarpışma merkezinden n adet düşük hasarlı mermi, 360° açısal dağılımla ($\frac{2\pi}{n}$) fırlatılır.
- **Seken Mermi (Bouncing Bullet):** Mermi bir engele çarptığında yok edilmek yerine, engelin normal vektörü ($\vec{n}$) üzerinden yansıma (Reflection) formülüyle yeni yönünü alır:

$$\vec{v}_{reflect} = \vec{v} - 2(\vec{v} \cdot \vec{n})\vec{n}$$

5.8 Hasar ve Respawn Mekanizması

Sunucu tarafında yönetilen hasar sistemi şu adımları izler:

- Çarpışma algılandığında mermi türüne göre hasar hesaplanır ve hedef tankın `health` puanından düşülür.
- Ölüm gerçekleştiğinde (`health` ≤ 0), saldırganın skoru güncellenir ve sunucu anında yeni bir rastgele harita şeması üreterek tüm oyuncuları "Yeni Tur" durumuna sokar.